

Smart Gauge Project Reference

End-to-end notes for the mobile app, backend, Strava connection, session security, and TDD workflow

Prepared as a teaching/reference document

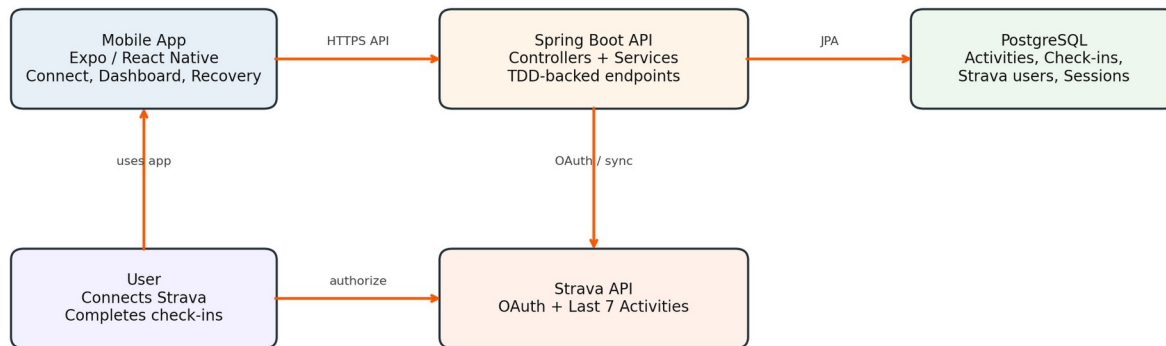
1. Executive Summary

Smart Gauge is a mobile-first endurance training and recovery application. The app connects to Strava, imports recent activities, asks the athlete to complete recovery check-ins, and builds a dashboard that summarizes training status, pending check-ins, pain, RPE, and activity history.

The project has evolved from a local TrainGuard prototype into a more beta-ready Smart Gauge architecture. The most important shift was moving from a hardcoded athleteId to a backend-issued Smart Gauge session token. This prevents beta testers from changing an athleteId in a URL and viewing another athlete's data.

- Frontend: Expo / React Native with Expo Router routes for Connect, Dashboard, Recovery, Analysis, and Strava callback handling.
- Backend: Spring Boot with controller-service-repository layering and TDD-first development.
- Database: PostgreSQL with activities, recovery check-ins, Strava users, and Smart Gauge sessions.
- Strava integration: OAuth authorization, backend callback, refresh-token storage, last-seven-activities sync, and dashboard rendering.
- Security improvement: frontend stores a session token, not athleteId; backend resolves the token to the correct athlete.

Smart Gauge - High-Level Architecture



2. What We Have Built So Far

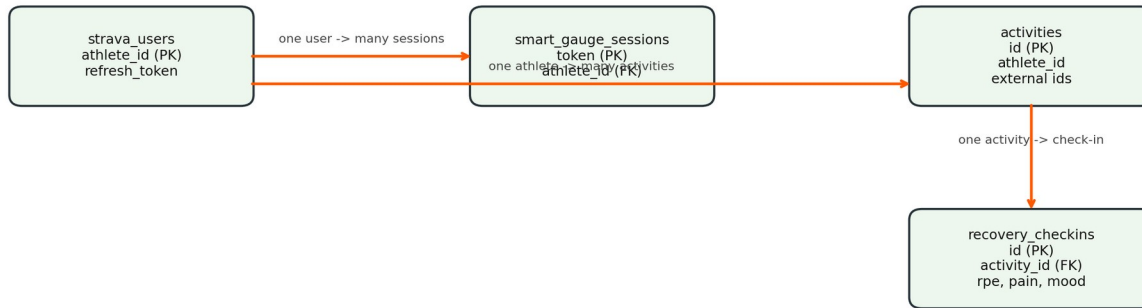
Area	Completed work	Why it matters
Activity backend	Activity entity, repository, import request/response records, ActivityMetricService, ActivityService, controller tests, and GET/POST endpoints.	This creates the core training activity model and converts raw Strava units into display-ready values.
Recovery backend	Recovery check-in entity, repository, service, controller, POST and GET endpoints.	This lets athletes report RPE, pain score, pain location, mood, and notes after activities.
Dashboard backend	Dashboard feed record, dashboard service, controller, and tests.	This combines activity data and recovery check-ins into one frontend-friendly feed.
Strava sync	Strava service/client records, OAuth URL generation, token exchange, last-seven-activity import, and duplicate handling through ActivityService.	This connects the app to real athlete activity data.
Session security	Smart Gauge session entity/service/repository and Authorization header-based API access.	This prevents a tester from requesting another athlete's data by editing an athleteId.
Frontend	Connect screen, dashboard, recovery check-in screen, dynamic tiles, bottom nav, logout, analysis placeholder, Strava redirect handling.	This gives the mobile app a usable MVP flow.

3. Project Architecture

The architecture follows a clear separation of responsibilities. The mobile app handles screens and user interaction. The backend owns OAuth, token refresh, Strava API calls, business logic, and data access. PostgreSQL stores long-lived data.

Layer	Main responsibility	Examples
Mobile app	Display data and collect user actions.	Connect with Strava, dashboard, recovery form, sync button, logout.
Controllers	Accept HTTP requests and delegate.	StravaController, DashboardController, RecoveryCheckinController.
Services	Business logic and orchestration.	StravaService, DashboardService, ActivityService, SessionService.
Clients	External API calls.	StravaClient calls Strava OAuth and activity endpoints.
Repositories	Database reads/writes.	ActivityRepository, RecoveryCheckinRepository, StravaUserRepository, SessionRepository.
Database	Persistence.	activities, recovery_checkins, strava_users, smart_gauge_sessions.

Core Data Relationships



4. Backend Feature Notes

4.1 Activity Import

Activity import is the central pipeline for creating activities. StravaService should not write activities directly to the repository. Instead, it converts Strava activities into `ActivityImportRequestRecord` and passes them to `ActivityService.importActivity()`.

```
StravaActivityResponseRecord
-> ActivityImportRequestRecord
-> ActivityService.importActivity()
-> ActivityEntity saved in PostgreSQL
-> ActivityResponseRecord returned to frontend
```

Important multi-user change: `ActivityImportRequestRecord` and `ActivityEntity` must include `athleteId`. This ensures every imported activity belongs to the correct Strava athlete.

```
public record ActivityImportRequestRecord(
    Long athleteId,
    String externalSource,
    String externalActivityId,
    String sportType,
    String name,
    LocalDateTime startDate,
    Double distanceMeters,
    Integer movingTimeSeconds,
    Integer elapsedTimeSeconds,
    Double totalElevationGain,
    Double averageHeartbeat,
```

```
        Double maxHeartbeat,  
        Double averageWatts,  
        Double weightedAverageWatts  
    ) {}
```

4.2 Recovery Check-In

Recovery check-ins are linked to activities. Each check-in captures the subjective athlete response after an activity.

- RPE: perceived effort after the activity.
- Pain score: 0-10 pain value.
- Pain location: free-text location such as hip, knee, adductor, etc.
- Mood: selected mood value.
- Note: optional context.

4.3 Dashboard Feed

The dashboard feed combines activity data and recovery data into one response so the frontend does not have to join arrays manually.

```
GET /api/dashboard-information  
Authorization: Bearer <sessionToken>
```

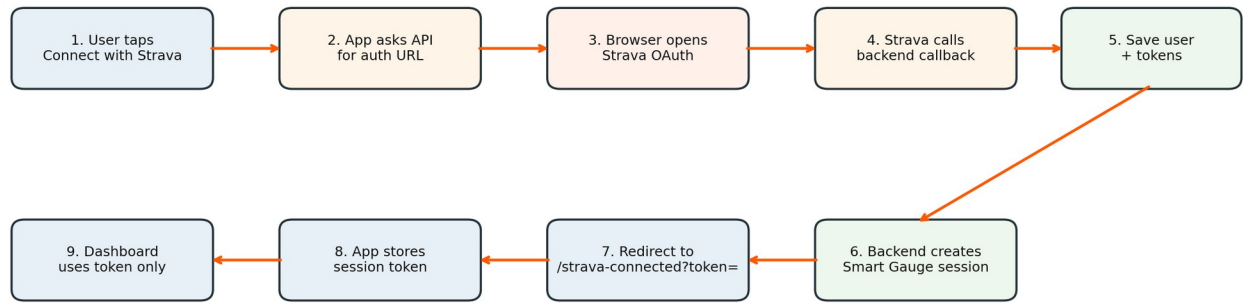
Backend:

1. Resolve session token to athleteId.
2. Fetch activities for that athlete only.
3. Find matching recovery check-ins.
4. Build DashboardFeedRecord list.
5. Return feed to mobile app.

5. Strava Connection Explained

The Strava connection is an OAuth flow. Smart Gauge never asks for or stores a Strava password. The athlete approves access on Strava's own page. Strava then gives the backend an authorization code, which the backend exchanges for tokens.

Strava Connect + Session Token Flow



5.1 OAuth URL Generation

The frontend should not manually build the Strava authorization URL. The backend builds it because the backend owns OAuth configuration such as `clientId`, `redirectUri`, and `scope`.

```
GET /api/strava/authorization-url
```

Returns plain text:

```
https://www.strava.com/oauth/authorize?
client_id=...&response_type=code&redirect_uri=...&approval_prompt=force&scope=read,activity:read_all
```

5.2 Why strava-connected.tsx Exists

`strava-connected.tsx` is a callback landing page for the frontend. It is not a normal user page. After Strava authorization, the backend redirects the browser/app to this route with a Smart Gauge session token.

Backend redirect after callback:

```
http://localhost:8081/strava-connected?token=<sessionToken>
```

Native app redirect later:

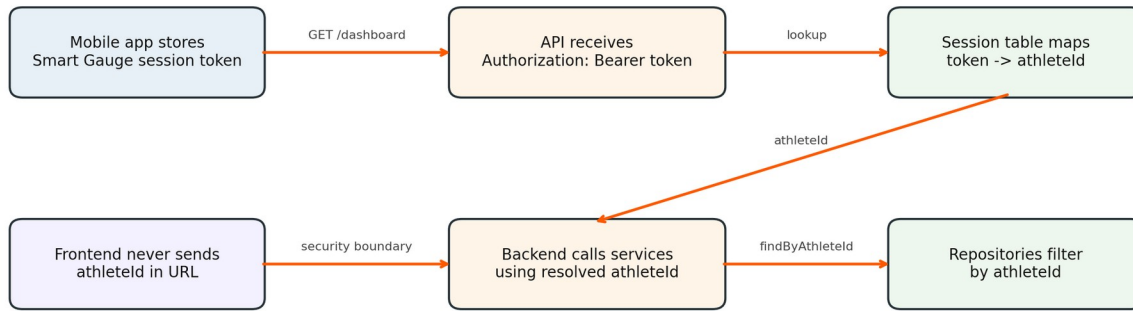
```
smartgauge://strava-connected?token=<sessionToken>
```

The route saves the token to `AsyncStorage` and then navigates to the dashboard. Without this route, the frontend has no clean place to receive and store the token.

6. Session Token Security Model

The original implementation used `athleteId` in frontend URLs. That was easy for local testing, but unsafe for beta testers. A user could technically change an `athleteId` and request another athlete's dashboard. The fix was to introduce Smart Gauge session tokens.

How Session Tokens Protect Athlete Data



6.1 Old Flow vs New Flow

Old athleteId flow	New session token flow
Frontend calls /dashboard-information/{athleteId}.	Frontend calls /dashboard-information with Authorization header.
Frontend knows and sends athleteId.	Frontend stores only a random session token.
A user could change the athleteId value.	Backend resolves athleteId from session token.
Not suitable for beta testers.	Acceptable MVP security for beta testing.

6.2 Session Token Request Flow

Frontend request:

GET /api/dashboard-information

Authorization: Bearer <smartGaugeSessionToken>

Backend:

1. Strip "Bearer ".
2. Find session by token.
3. Check expiration.
4. Get session.stravaUser.athleteId.
5. Call DashboardService with athleteId.
6. Return only that athlete's data.

7. API Endpoint Reference

Endpoint	Method	Auth	Purpose
/api/strava/authorization-url	GET	No session required	Returns the Strava OAuth URL to open.
/api/strava/callback?code=...	GET	Called by Strava	Exchanges code, saves Strava user, creates session, redirects to

			frontend with token.
/api/strava/sync	POST	Authorization: Bearer <token>	Syncs last seven Strava activities for the session's athlete.
/api/dashboard-information	GET	Authorization: Bearer <token>	Returns dashboard feed for the session's athlete.
/api/recovery-checkins	POST	Should use session later	Creates a recovery check-in.
/api/recovery-checkins	GET	Should use session later	Lists recovery check-ins.

8. Frontend Notes

8.1 Main Routes

Route	File	Purpose
/	src/app/index.tsx	Connect/Login page. Starts the Strava connection flow.
/strava-connected	src/app/strava-connected.tsx	Receives session token after backend callback redirect.
/dashboard	src/app/dashboard.tsx	Loads dashboard feed and handles Sync.
/recovery/[activityId]	src/app/recovery/[activityId].tsx	Recovery check-in form.
/analysis	src/app/analysis.tsx	Placeholder for future analysis features.

8.2 Session Storage

The frontend uses AsyncStorage to persist the Smart Gauge session token. It no longer stores athleteId as the primary access key.

```
const SESSION_TOKEN_KEY = "smart_gauge_session_token";
```

```
saveSessionToken(token)
```

```
getSessionToken()
```

```
clearSessionToken()
```

8.3 Sync Button

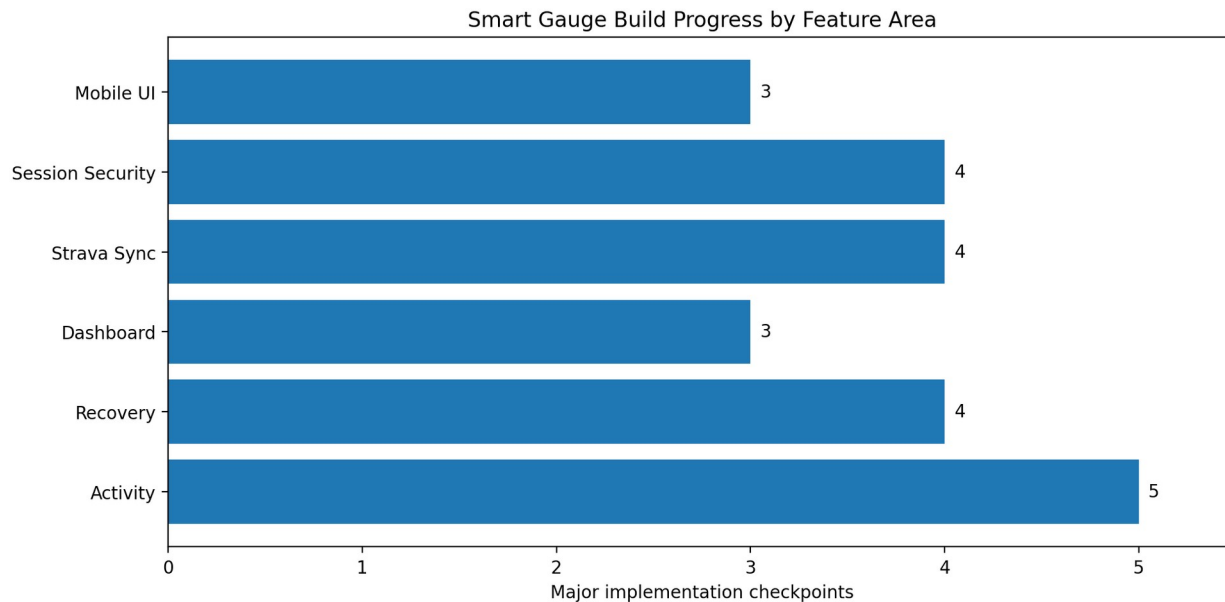
The Sync button does not send athleteId. It calls the backend with the session token in the Authorization header.

```
POST /api/strava/sync
```

```
Authorization: Bearer <sessionToken>
```

9. TDD Progression and Teaching Notes

The project followed a controller-service-repository pattern with TDD. Controller tests proved routing, status codes, JSON responses, and service delegation. Service tests proved business logic while mocking dependencies. Repository/database behavior was verified manually and can later be covered with integration tests.



Test type	What it proves	What it does not prove
Controller test	Endpoint path, HTTP method, response JSON, and that the correct service method is called.	Real service logic, repository logic, external API calls.
Service test	Business logic, mapping, duplicate handling, session-token resolution, and orchestration.	HTTP routing or real database behavior.
Client test/manual check	External API call behavior and Strava response handling.	Business logic unless isolated with mocks.
Manual request.http test	End-to-end local behavior through the backend.	Not automated; can miss regressions.

10. Troubleshooting Notes

Problem	Likely cause	Fix
Strava says redirect_uri invalid	STRAVA_REDIRECT_URI was not resolved or callback domain mismatch.	Load .env correctly and set Strava callback domain to localhost or hosted backend domain.
Authorization URL contains \${STRAVA_REDIRECT_URI}	Spring did not load environment variables.	Run set -a; source .env; set +a; ./gradlew bootRun from same terminal.
smartgauge:// opens broken page in browser	Custom scheme only works for installed/native app.	For Expo web use http://localhost:8081/strava-connected.
No static resource api/strava/null	App redirect property was null and backend redirected to null.	Set SMART_GAUGE_APP_REDIRECT_URI and enable its properties class.
athlete_id null in activities	athleteId not passed through import request or not set in ActivityEntity builder.	Add athleteId to ActivityImportRequestRecord and .athleteId(request.athleteId()) in saveNewActivity.
Controller test ApplicationContext fails	New controller dependency was not mocked in @WebMvcTest.	Add @MockitoBean for SessionService or SmartGaugeProperties.

11. Beta Hosting Direction

For beta testers, local localhost URLs must be replaced with hosted URLs. Render can host the Spring Boot backend and PostgreSQL for a beta. The mobile app should be distributed with TestFlight or a platform-appropriate beta channel.

- Backend on Render with HTTPS URL.
- PostgreSQL on Render or another managed provider.
- Environment variables set in Render dashboard, not committed to Git.
- Strava callback domain updated to the Render backend domain.
- Mobile API_BASE_URL changed from localhost to the Render backend URL.
- Native app redirect uses smartgauge://strava-connected?token=... for TestFlight builds.

12. How to Explain the System to Others

A simple teaching script:

1. Smart Gauge does not ask for a username or password. It uses Strava as the connection point.
2. When the athlete taps Connect with Strava, the app asks our backend for a Strava authorization URL.
3. The athlete approves access on Strava. Strava sends our backend a temporary code.
4. The backend exchanges that code for Strava tokens and saves the athlete's refresh token securely in PostgreSQL.
5. The backend creates a Smart Gauge session token and sends it back to the app.
6. The app stores only that Smart Gauge session token, not the athleteId or Strava password.
7. When the app asks for dashboard data or syncs activities, it sends the session token.
8. The backend resolves the session token to the correct athlete and only returns that athlete's data.

13. Recommended Next Steps

- Make recovery check-in endpoints session-token aware so check-ins cannot be created against another athlete's activity.
- Add a backend disconnect endpoint that deletes or invalidates sessions and optionally removes stored refresh token.
- Add session expiration handling in the frontend.
- Add API error response objects instead of raw exceptions.
- Add Render deployment configuration and production environment variables.
- Move app API base URL into environment-specific mobile config.
- Add a real analysis screen using dashboard and recovery data trends.